Assignment 5 -CO5

NAME	GAYATHRI.P
REG NO	192424324
COURSE	Mobile Computing
CODE	ITA0329

PART A – DESIGN, ANALYSIS & ARCHITECTURAL REASONING

TASK A1 – Constraint-Aware Intent & Service Architecture

A1.1 Model the Problem

Element	Healthcare-monitoring app
Input	Sensor events (heart-rate, SpO_2, glucose) with timestamp and severity; network state (none / metered-poor / unmetered-good); battery level and charging flag; Doze / App-Standby state; user preferences (Wi-Fi-only, quiet hours); permission and consent state; login session
Output	Encrypted records uploaded to backend (acknowledged by server); local notifications for critical events; audit log of deferred / cancelled work
Decision at each stage	(1) Collect now or batch? (2) Which pending item next? (3) Which component runs it: foreground Service, WorkManager worker, or high-priority alert path? (4) Start now, defer, or cancel? (5) Notify or stay silent?

Constraints	Battery floor (normal ≥ 20 %, critical ≥ 5 %); Doze / App-Standby (only maintenance windows or high-priority alerts); network required; runtime permission + consent; data must be encrypted (AES-GCM, keys in Keystore) before leaving memory; no PHI in notification text or logs; max concurrent workers; main thread must stay free (< 16 ms frames)
Goal condition	Every record that is permitted is either delivered exactly once (server ACK) or is still safely stored encrypted for a later window; every critical event produces a user notification within its deadline (3 ticks in the simulation); no plaintext PHI is exposed; no work is left running after logout / permission revoke

A1.2 Runtime State

While a task is partially executed or waiting for network we must keep:

State = ⟨ Q, W, E, S, C ⟩

- **Q** – pending queue; each item = ⟨id, arrival, priority, size, deadline, sensitive?, encrypted?, attempts, uploadedBytes, serverToken⟩
- **W** – set of running workers (id → item, start time, cancellation handle)
- **E** – environment snapshot ⟨net, battery, charging, doze, permission, loggedIn, maintenanceWindow⟩
- **S** – security context ⟨key alias, encryption status per item, session token⟩
- **C** – counters / log ⟨enqueued, deferred, cancelled, succeeded, peak workers⟩

The fields uploadedBytes and serverToken (idempotency key) allow resumable upload, so a task interrupted by Doze or network loss resumes rather than restarts and a retry cannot create duplicate server records.

A1.3 Pruning / Feasibility Condition

A scheduled task *t* may **start** only if

FEASIBLE(t, E) = perm ∧ loggedIn ∧ net ≥ 1 ∧ size(t) ≤ bandwidthBudget(net) ∧ battery ≥ floor(prio(t)) ∧ (¬doze ∨ prio(t)=CRITICAL ∨ maintenanceWindow) ∧ (¬sensitive(t) ∨ encrypted(t))

Decision rules when FEASIBLE is false:

- **Defer** (safe, correctness preserved) when the violated condition is *temporary*: network, battery, Doze, bandwidth budget. The item stays in Q (still encrypted at rest).
- **Cancel / purge** when the violated condition is *terminal*: permission revoked, user logged out, consent withdrawn, or item past a retention limit. Data is wiped and in-flight workers are rolled back.
- **Reject** when the item can never be made safe: sensitive but encryption failed (never upload plaintext).

Deferral is always safe because nothing is lost; cancellation is safe only for terminal conditions; a critical item is never deferred because of battery above its 5 % floor or Doze.

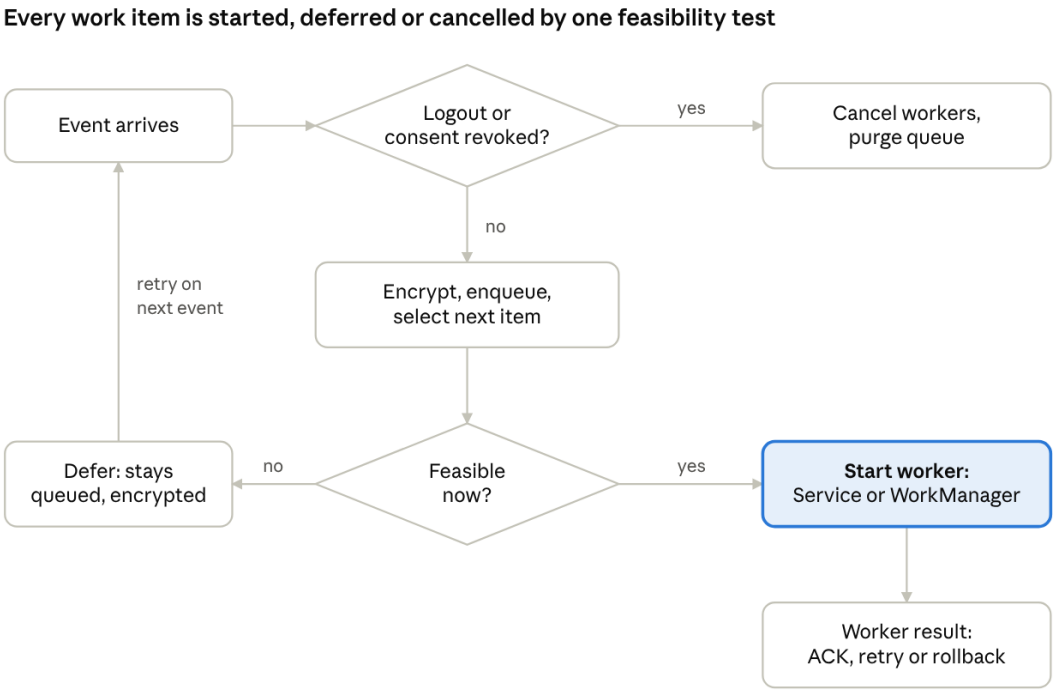
A1.4 Complete Architectural Pseudocode

```
ALGORITHM BackgroundScheduler(Q, E_stream, MAX_W)
  W ← ∅ ; C ← counters(0)
  ON EVENT e FROM E_stream DO
    E ← update_environment(E, e)
    IF e is LOGOUT or PERMISSION_REVOKED THEN
       CANCEL_ALL(W)                    // undo: rollback partial uploads
       PURGE(Q) ; C.cancelled += |Q| ; CONTINUE
    IF e is SENSOR_READING THEN
       item ← ENCRYPT(make_item(e))       // encrypt before enqueue
       Q ← Q ∪ {item} ; C.enqueued += 1
       IF item.priority = CRITICAL THEN
          send EXPLICIT_INTENT(AlertReceiver, item.id)  // user notification path
    SCHEDULE_TICK()
  END ON
```

```
PROCEDURE SCHEDULE_TICK()
  budget ← bandwidthBudget(E.net)
  started ← ∅
  WHILE Q ≠ ∅ AND |W| < MAX_W DO
    t ← SELECT_NEXT(Q)                    // priority, then deadline, then smaller size
    comp ← CHOOSE_COMPONENT(t)            // CRITICAL → ForegroundService (explicit
Intent)
BATTERY_NOT_LOW)
    IF FEASIBLE(t, E) AND size(t) ≤ budget THEN
       ENQUEUE_AND_START(comp, t)         // starts worker, budget −= size(t)
       W ← W ∪ {t} ; Q ← Q \ {t}
    ELSE IF TERMINAL_VIOLATION(t, E) THEN
       DISCARD(t) ; C.cancelled += 1
    ELSE
       DEFER(t) ; C.deferred += 1         // stays in Q; WorkManager re-evaluates on constraint
change
       MARK_SKIPPED(t)
    IF no candidate left that is not MARKED THEN BREAK
  END WHILE

ON WORKER_RESULT(t, r) DO
  IF r = SUCCESS(ack)  THEN W ← W \ {t} ; C.succeeded += 1 ; DELETE_LOCAL(t)
  IF r = RETRYABLE     THEN W ← W \ {t} ; t.attempts += 1 ; Q ← Q ∪ {t} ; BACKOFF(t)
  IF r = FATAL         THEN ROLLBACK(t) ; W ← W \ {t} ; AUDIT(t)
END ON

SUCCESS: Q = ∅ ∧ W = ∅ ∧ every critical item acknowledged within deadline
FAILURE: terminal violation for any item with undelivered critical data → raise local alert
```



A1 scheduler flow · 2 decisions, 3 outcomes

A terminal event cancels everything; otherwise one feasibility test either starts the worker or defers the item, and deferred items return on the next event.

Intent types used: *explicit* Intent for SensorService → UploadWorker and AlertReceiver (stays inside our package, so no data leaks to other apps); *implicit* Intent only for system broadcasts (CONNECTIVITY_CHANGE, ACTION_BATTERY_LOW) received through a registered receiver; *ordered* broadcast for the critical-alert chain (app receiver first, can abort the broadcast after handling so lower-priority receivers never see PHI). All explicit Intents use PendingIntent.FLAG_IMMUTABLE.

A1.5 Trace of the Design (5 distinct events)

Scenario produced by running the scheduler (priority policy) on the scripted event list:

Tick	Event	State / constraint check	Action
0	**Sensor reading** R0 (normal,	FEASIBLE ✓	CHOOSE R0 → execute →

	size 2); net good, battery 80 %		uploaded
1	R1 reading (normal, size 4); **connectivity lost** (net = 0)	net ≥ 1 ✗	R1 **deferred** (stays encrypted in Q)
2	**Critical alert** R2 (size 1, deadline 5); **Doze entered**; still offline	critical bypasses Doze, but net ✗	R2 deferred; AlertReceiver notifies the user locally
3	Connectivity restored; R3 (high, size 3) arrives; Doze on, not a maintenance window	R2: critical ✓ net ✓ → run; R1, R3: Doze ✗	R2 **executed** (delay 1 tick, within deadline) → server ACK; R1, R3 deferred
4–5	R4 arrives; Doze persists	R1, R3, R4 non-critical, Doze ✗	all deferred (11 deferral decisions in total)
6	**User logout**	terminal violation	**Cancel / backtrack**: workers rolled back, R1, R3, R4 purged, counters: enqueued 5, succeeded 2, cancelled 3

Choose → Check Constraint → Execute/Defer → Notify → Cancel/Backtrack is exercised: critical data was delivered on time, no plaintext left the device, and nothing survived the logout.

TASK A2 – Don't Synchronise What Cannot Be Safe

A2.1 Diagnose the Waste

`SyncAll` uploads every record in storage order and merely `CONTINUE`s on failure.

- **Unnecessary work / energy:** records are retried with no back-off and re-sent in full after every failure (no delta, no resume), so the radio is repeatedly woken (radio tail energy) on a dead link; large low-value records are uploaded before small critical ones.

- **Battery blindness:** the loop keeps draining the battery at 5 % exactly as at 100 %.
- **No conflict handling:** local records overwrite newer server versions (lost updates) and duplicates are created because there is no idempotency key.
- **Security leakage:** records are uploaded whether or not they are encrypted; there is no check of the connection (cleartext HTTP or untrusted Wi-Fi) or of consent; failed attempts may be logged with PHI; nothing stops uploads after logout.
- **Decision latency:** all choices are postponed until after the whole store is scanned; nothing is pruned early.

A2.2 Incremental, Constraint-Aware Redesign

```
ALGORITHM SyncIncremental(LocalStore, RemoteAPI, E)
  S ← ∅ (sync set) ; D ← ∅ (deferred) ; X ← ∅ (discarded / merged)
  order ← SORT(LocalStore by (priority DESC, dirty-since ASC, size ASC))
  FOR each record R in order DO
    // ---- pruning tests (cheapest first) ----
    IF NOT E.permission OR NOT E.loggedIn          THEN X ← X ∪ {R}; CONTINUE       //
terminal
    IF R.sensitive AND NOT R.encrypted             THEN D ← D ∪ {R}; ENCRYPT_LATER(R);
CONTINUE
    IF E.battery < floor(R.priority)               THEN D ← D ∪ {R}; CONTINUE       // all lower
priorities also fail
    IF quality(E.net) < need(R) OR R.size > budget THEN D ← D ∪ {R}; CONTINUE
    IF NOT E.secureChannel                         THEN D ← D ∪ {R}; CONTINUE       // TLS +
pinned cert
    // ---- alternatives: Upload | Defer | Discard/Resolve ----
    remote ← RemoteAPI.HEAD(R.id)                  // version / ETag
    IF remote.version > R.baseVersion THEN
      R' ← RESOLVE(R, remote)                      // merge by field / user rule
      IF R' = ∅ THEN X ← X ∪ {R}; CONTINUE ELSE R ← R'
    S ← S ∪ {R} ; budget ← budget − R.size ; E.battery ← E.battery − cost(R)
    IF E.battery < floorMin THEN BREAK             // monotone: no later record can be feasible
```

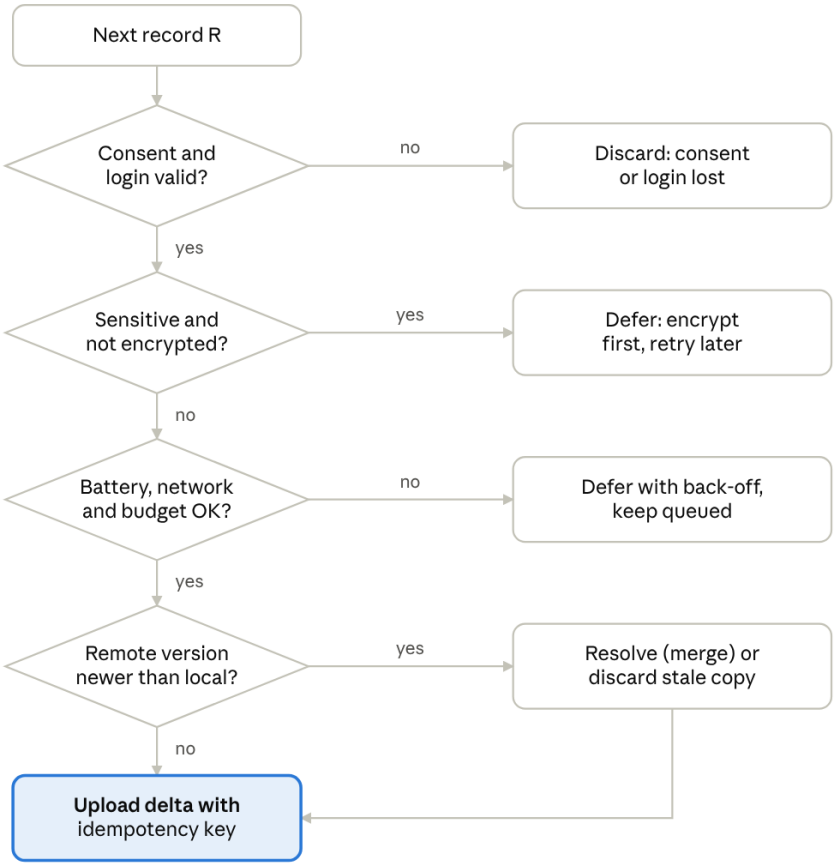
END FOR

FOR each R in S: UPLOAD_DELTA(R, idempotencyKey = R.id+version); on failure → D ← D ∪ {R} with backoff

RETURN (S, D, X)

END

Each record is tested cheapest-first and ends as upload, defer or discard



A2 per-record sync decision · 4 tests, 3 outcomes

The tests run cheapest first, so a record that fails a terminal or security test never reaches the network; only a record that passes all four is uploaded.

A2.3 Demonstrate Pruning (five records)

Condition: battery 12 %, metered poor network (≈ 3 size units per tick), consent valid.

Record	Priority	Size	Sensitivity	Status
R1	3 (critical alarm)	1	sensitive, encrypted	feasible (floor 5 %)
R2	1 (bulk log)	40	none	pruned: size > budget and battery < 20 %
R3	2 (vitals)	2	sensitive, **not encrypted**	pruned: security rule (defer until encrypted)
R4	1 (photo)	5	none	pruned: battery < 20 %, conflict pending
R5	2 (vitals)	2	sensitive, encrypted	pruned: battery 12 % < floor 20 %

Early termination: after R1 is uploaded the battery is ≈ 11.9 %. The branch "upload any of R2–R5" is cut at once. Battery only decreases while unplugged and every remaining record has a floor of 20 %, so the condition battery ≥ 20 % cannot become true inside this sync session. Exploring that sub-tree could only produce a sync set that violates the battery constraint (or, for R3, the security constraint), so no correct or safe synchronisation exists in it. R2–R5 move to D.

A2.4 Challenge the Pruning Rule

With **offline-first and eventual consistency** the *network, battery and Doze* prunes remain valid as **deferrals** (they only delay). Two assumptions break:

1. The *conflict rule* "if remote.version > local then resolve or discard" assumed the server is the single authority at the time of the sync. Under offline-first, divergent concurrent updates are normal; discarding a record loses data. The violated assumption is **"a stale local record is obsolete"**. We must keep it and merge with vector clocks / CRDT / field-level last-writer-wins.
2. The "terminate the loop early" step assumed a **single sync session must finish all records**. In offline-first the remaining records are retried in later windows, so termination

means *stop this attempt*, not *give up*. The pruning cannot discard records, only postpone them.

TASK A3 – Discover the Mobile-Background Design Pattern

Design component	**Sensor collection**	**Data sync**	**Push notification**
State	sensor buffer, sampling rate, battery, permission	pending records, versions, network, battery, encryption flags	message queue, device token, user presence / DND, notification channel
Decision variable	sampling interval and batch size	which record to send next, batch size	whether and how to alert (silent, banner, full-screen)
Candidate choices	sample now / reduce rate / batch / skip	upload / defer / merge / discard	show now / delay / collapse / drop
Feasibility / constraint test	permission ✓, battery above floor, sensor available, not in quiet hours	consent ✓, net quality, battery, record encrypted, secure channel	notification permission ✓, channel enabled, rate limit, no PHI in text
Goal condition	all required samples captured at the required rate	every record acknowledged by server or safely deferred	every critical event shown within deadline, no duplicates
Pruning / deferral condition	battery low or Doze → lower rate (monotone)	battery < floor or net too poor → defer	user in DND or limit exceeded → collapse or delay (critical bypass)
Undo /	stop sampling, free	roll back partial	cancel the pending

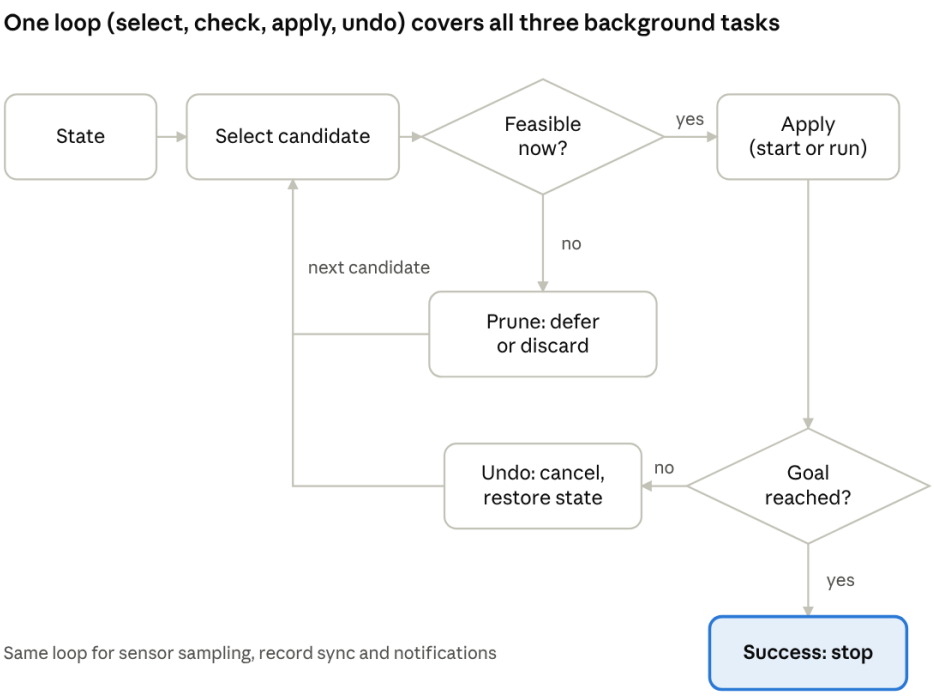
cancel	sensor, discard partial buffer	upload, release lock, remove temp copy	notification and remove stale alerts

A3.1 Common Pattern

All three are **constrained sequential decision problems over a queue of work items**: at each step pick one candidate, test it against a monotone set of resource and security constraints, commit (execute) or prune (defer / discard), update state, and undo if a later event invalidates the choice. Computationally this is *incremental construction with feasibility pruning and backtracking* (a bounded search / scheduling pattern), architecturally an *event-driven loop* of **State → Select → Check → Act → Update → Undo**. They differ only in the resource being protected (sensor energy, radio and data, user attention) and in the item type.

A3.2 Generic Framework

```
PROCEDURE SCHEDULE_OR_EXECUTE(State)
  IF GOAL(State) THEN RETURN SUCCESS
  IF TERMINAL_VIOLATION(State) THEN CLEANUP(State); RETURN FAILURE
  FOR each candidate c IN ORDER(CANDIDATES(State)) DO     // rule: priority, deadline, cost
     IF NOT FEASIBLE(State, c) THEN
        IF PERMANENTLY_INFEASIBLE(State, c) THEN DISCARD(c) ELSE DEFER(c)
        CONTINUE                                  // prune this branch
     State' ← APPLY(State, c)                     // start worker / sample / notify
     result ← SCHEDULE_OR_EXECUTE(State')         // or wait for the event
     IF result = SUCCESS THEN RETURN SUCCESS
     State ← UNDO(State', c)                      // cancel, release, roll back
  END FOR
  RETURN WAIT_FOR_NEXT_EVENT(State)               // deferred work retried when
constraints change
END
```



A3 generic background-work loop · 2 decisions, 2 return paths

Both the prune path and the undo path return to candidate selection, which is why one framework fits sampling, sync and notifications.

Instantiation: *Sensor* → candidates = sampling modes; *Sync* → candidates = records; *Notification* → candidates = messages and presentation modes. Only FEASIBLE, APPLY, UNDO and GOAL change.

A3.3 Role of Undo / Cancellation

A background framework runs many partially-completed operations; if a task is cancelled or fails without restoring state, the system is left inconsistent and insecure. Example (data sync): a 5 MB record is half-uploaded when the user logs out. Without rollback, (a) the temp encrypted copy and key stay on disk (privacy leak), (b) the record is marked "sent" so it is never retried (data loss), (c) the half-written server object stays visible (corruption), (d) the worker keeps a wake-lock and drains the battery. Undo returns State to a consistent point so that retry or cleanup is correct.

TASK A4 – Tractability: Local vs Cloud vs Hybrid

#	Problem	What constitutes a solution?	Can a design be verified efficiently?	Resource class	Reason
1	On-device image classification (tiny model)	A label for the image from a quantised network	Yes – a forward pass is polynomial (milliseconds) and checkable against a validation set	**On-device feasible**	Small model fits in RAM; no network; NPU/GPU delegate; privacy retained
2	Cloud speech recognition	Transcript text meeting an accuracy target	Yes – WER computed on a test set, but needs the server	**Network-bound** (Hybrid if wake-word on device)	Large model, audio upload and latency dominate; scalability by cloud autoscaling
3	P2P file sharing among nearby devices	Complete, intact file transfer with verified peers	Yes – hash and signature verify the file, peer authentication check	**Battery-critical**	Radio discovery, Wi-Fi Direct / BLE scanning and transfer drain battery; no server needed
4	End-to-end encrypted messaging	Message readable only by sender and receiver	Yes – protocol verification and key checks are efficient, but breaking is hard	**Security-critical**	Cryptography and key management on device; server only relays ciphertext
5	Real-time multiplayer	All clients converge to a	Partially – consistency checks	**Hybrid required**	Authoritative server plus client prediction

	game state sync	consistent state within ~50–100 ms	are cheap, latency guarantees are not verifiable offline		and interpolation; network jitter and battery both matter
6	Large-scale offline map rendering with vector tiles	Smooth 60 fps map with correct labels from local tiles	Yes – frame time and tile correctness are measurable	**On-device feasible** (battery-critical for GPU)	Tiles pre-downloaded; GPU rendering with caching; memory limits dominate

Inquiry 1 – Why is "runs slowly on a phone" inadequate? It does not say *what* is slow (frame time, start-up, network round-trip), *where* the time goes (CPU, GPU, I/O, radio), *under which conditions* (device class, thermal throttling, battery, signal) or *against which target* (16 ms frame, 100 ms interaction). A performance problem must be a measurable property with a metric, a threshold, a workload and a cause, otherwise it cannot be diagnosed, compared or verified after a fix. It also hides energy and memory costs: a fast but battery-draining design is still a failure.

Inquiry 2 – Why does local-vs-cloud matter for scalability and security? Local processing scales with the *number of devices* (each brings its own compute, no server cost) but is limited by each device's CPU, memory and battery and by device heterogeneity. Cloud offloading scales with *server capacity* and gives uniform model quality but adds network latency, bandwidth cost, per-user server cost and a *new attack surface* (data in transit and at rest, third-party access). Local keeps sensitive data on the device; cloud requires encryption, consent and trust in the provider. Interpreting a benchmark or threat model therefore depends on where the work runs, which is why hybrid designs split work by sensitivity and cost.

TASK A5 – Design When Exact Optimisation Is Too Expensive

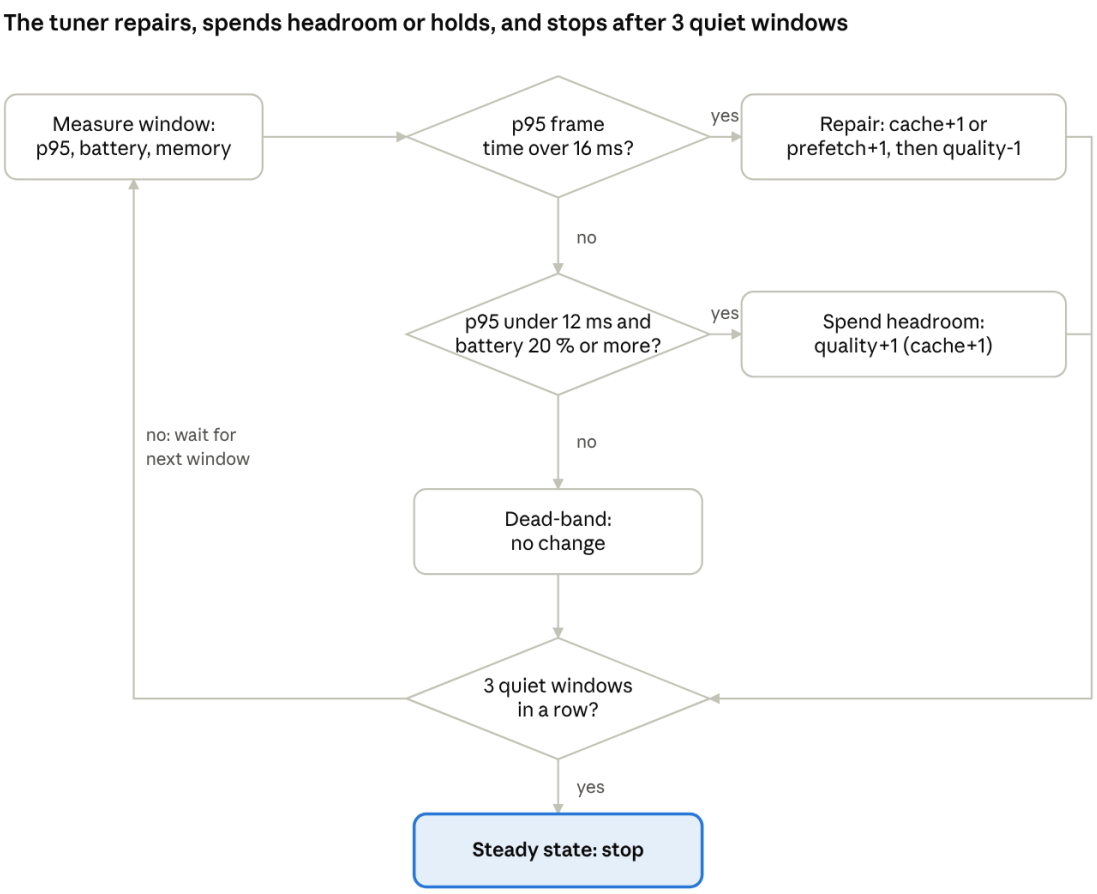
A5.1 Exact Perspective

Let Q decoding resolutions, C cache sizes and P prefetch distances. An exact method enumerates every configuration (q, c, p), runs or simulates the feed for each, and keeps the one that

minimises cost = w_1·droppedFrames + w_2·energy + w_3·qualityLoss. For a *single* global configuration this costs |Q|·|C|·|P| simulations. It becomes problematic because (a) per-item or per-class settings multiply the space: $Q^K \times C \times P$ for K item classes, (b) every device profile (CPU, GPU, RAM, battery) needs its own search, (c) each evaluation requires scrolling through N feed items, so total work ≈ $Q^K \cdot C \cdot P \cdot N \cdot D$ for D device profiles, and (d) the optimum changes as battery, thermal state and content change, so a result computed offline is stale. On a phone the search itself would blow the frame and energy budget it is meant to protect.

A5.2 Adaptive Greedy Pseudocode

```
PROCEDURE AdaptiveFeedTuner(device)
  params ← (q = mid, cache = c0, prefetch = p0)       // selected quality / cache parameters
  REPEAT every window of 30 frames
    M ← MEASURE(frameTime_p95, droppedFrames, batteryLevel, memoryPressure)   // current metrics
    IF M.frameTime_p95 > 16 ms THEN                   // selection criterion: best ms-saved per extra energy
        a ← argmax over {cache+1, prefetch+1, quality−1} of Δms / Δenergy
        IF battery < 20 % OR memoryPressure THEN prefer quality−1 and cache−1
    ELSE IF M.frameTime_p95 < 12 ms AND battery ≥ 20 % THEN
        a ← quality+1 (optionally with cache+1)       // spend headroom
    ELSE a ← NONE                                     // dead-band prevents oscillation
    IF a ≠ NONE THEN params ← APPLY(params, a)        // state update (evict LRU if cache shrinks)
  UNTIL STEADY: no action for 3 consecutive windows OR user stops scrolling
```



A5 adaptive feed tuner · 3 decisions, 1 stop condition

Each 30-frame window ends in one of three actions; a change resets the quiet-window count, and three quiet windows in a row end the loop.

Maintained explicitly: **metrics** (M), **parameters** (params), **selection criterion** (Δms / Δenergy, battery override), **state update** (APPLY and eviction), **termination** (steady state with dead-band 12–16 ms).

A5.3 What Have We Sacrificed?

The greedy loop finds a **feasible** configuration (p95 ≤ 16 ms) quickly by following local signals, but it never compares it with all alternatives. It cannot prove that the chosen parameters **minimise total energy for a given quality of service**; a different combination (for example a larger cache with a lower resolution) might have the same smoothness at lower energy. We give

up the *optimality guarantee* in exchange for a bounded, tiny decision cost (a handful of evaluations per window), adaptability to changing conditions and no offline search per device.

PART B – IMPLEMENTATION & COMPUTATIONAL INVESTIGATION

How the evidence was produced. Parts B1–B4 are implemented as deterministic discrete-time simulations in Python (ita03_ca5_simulations.py, fixed seeds, run it to reproduce every number below) plus a Kotlin WorkManager reference implementation of the B1 design. The numbers are simulated (battery, network and Doze states are scripted), not measured on a physical phone. If your instructor requires on-device figures, run the Kotlin worker on an emulator and read the same counters from Metrics.

TASK B1 – Implement and Instrument the Background Architecture

Implementation. The simulator runs 20 sensor records (3 critical, 5 high, 12 normal; sizes 1–6 units) over 30 ticks with at most **3 concurrent workers** and a per-tick upload budget of 8 units on good network, 3 on poor network, 0 offline. Feasibility is the A1.3 rule: permission, network, battery floor (critical 5 %, others 20 %), Doze (only critical or the maintenance window every 6th tick), bandwidth budget. Poor-network uploads fail deterministically one time in four and are retried. Recorded for every run: items enqueued, deferral events, distinct items deferred, cancelled, successes, retries, peak concurrent workers, energy, final backlog and critical items on time.

Android reference (the real-device counterpart of the pseudocode):

```
class UploadWorker(ctx: Context, p: WorkerParameters) : CoroutineWorker(ctx, p) {
  override suspend fun doWork(): Result {
    val id = inputData.getLong("recordId", -1)
    if (!ConsentStore.isGranted(applicationContext)) { Repo.purge(id); return Result.failure() } //
terminal
    val rec = Repo.loadEncrypted(id) ?: return Result.success()
    return try {
      Api.upload(rec, idempotencyKey = "$id-${rec.version}")   // resumable + idempotent
```

```
      Repo.markDelivered(id); Metrics.succeeded.incrementAndGet(); Result.success()
    } catch (e: IOException) {
      Metrics.retries.incrementAndGet()
      if (runAttemptCount < 5) Result.retry() else Result.failure()
    }
  }
}
fun enqueue(ctx: Context, rec: Record) {
  Metrics.enqueued.incrementAndGet()
  val c = Constraints.Builder().setRequiredNetworkType(NetworkType.CONNECTED)
          .setRequiresBatteryNotLow(true).build()
  val b = OneTimeWorkRequestBuilder<UploadWorker>().setConstraints(c)
          .setInputData(workDataOf("recordId" to rec.id))
          .setBackoffCriteria(BackoffPolicy.EXPONENTIAL, 30, TimeUnit.SECONDS)
  if (rec.priority == CRITICAL)
b.setExpedited(OutOfQuotaPolicy.RUN_AS_NON_EXPEDITED_WORK_REQUEST)
  WorkManager.getInstance(ctx).enqueueUniqueWork("rec-${rec.id}",
ExistingWorkPolicy.KEEP, b.build())
}
fun onLogout(ctx: Context) { WorkManager.getInstance(ctx).cancelAllWork(); Repo.purgeAll()
}
```

B1.1 Validate the Implementation

Test	Scenario	What it verifies	Result (priority scheduler)
T1 Success path	S1: good network every tick, battery 80 %→65 %, no Doze	Correct delivery when no constraint binds; baseline energy	20/20 delivered, backlog 0, 3/3 critical on time, energy 93
T2 Defer / cancel under	S2: network good→poor→offline →good; Doze ticks 6–	Deferral, not loss; Doze bypass for critical only; recovery	18/20 delivered, 2 still queued (encrypted), 3/3 critical on time, 0 data lost, 175 deferral

Doze + low battery	17; battery falls from 60 % to the 3 % floor by tick 22, plugged in at tick 24	in the later window	events over 14 distinct items
T3 Boundary: permission revoked mid-run	S3: network drops to poor at tick 8, consent revoked at tick 12	Terminal violation purges the queue; later readings are rejected, nothing uploaded without consent	13 enqueued, 7 rejected after revoke, 1 queued item purged (cancelled), 12 delivered before revoke, no worker still running after revoke

B1.2 Compare Design with Implementation

Changes made while implementing, and why:

- **Deferral counted per tick as an event**, in addition to the number of distinct items. The pseudocode counted once per decision; a queue waiting through Doze produces one decision per tick, and that repeated work is exactly the cost we want to see.
- **Bandwidth budget added to the feasibility test.** The A1.3 rule had only a network flag; without a per-tick budget two large uploads could start together on a poor link and both fail.
- **Deterministic transient failures on poor links** were added so that retries and wasted energy are visible.
- **Doze maintenance window** modelled as every 6th tick, because real Doze lets deferred work run only in those windows.
- **Order of checks:** logout and permission are tested first, before selection, as in ON EVENT; the in-flight rollback path exists in the code, but in S3 no worker was running at the revoke tick, so the rollback count is 0 (the purge of the queue did occur).
- **WorkManager constraints replace part of the manual test.** On a device setRequiredNetworkType and setRequiresBatteryNotLow perform the network and battery check for deferred work; our own check remains for priority, consent and encryption.

B1.3 Improve the Scheduling Order

Version 1 (FIFO). Items are taken in arrival order. It is strict: if the head item cannot start, the items behind it wait (head-of-line blocking), as in a single sequential queue.

Version 2 (priority heuristic). Candidates are sorted by (priority DESC, deadline ASC, size ASC, arrival ASC) and each is started if it passes the feasibility test and fits the remaining budget; infeasible candidates are skipped, not waited on.

Rationale: critical items have a deadline, so they go first; among equals the earliest deadline avoids a miss; the smallest size first lets many items fit in a small poor-network budget (shortest-job-first), and skip-and-fit removes head-of-line blocking.

B1.4 Measure the Effect

Same 20-item workload, same environment. V1 = FIFO, V2 = priority. Values in brackets are distinct items deferred.

Measure	**S1 V1**	**S1 V2**	**S2 V1**	**S2 V2**	**S3 V1**	**S3 V2**
Work items enqueued	20	20	20	20	13	13
Deferral events (distinct items)	9 (8)	6 (4)	200 (18)	175 (14)	9 (8)	6 (4)
Cancelled	0	0	0	0	1	1
Successful executions	20	20	17	18	12	12
Peak concurrent workers	3	3	3	3	3	3
Final backlog	0	0	3	2	0	0
Critical items on time	3/3	3/3	2/3	3/3	2/3	2/3
Mean critical latency (ticks)	0.0	0.0	5.0	0.67	0.0	0.0
Energy (units)	93	93	78	80	53	53

Interpretation. The ordering rule did **not** reduce the number of items enqueued, the peak number of workers or the total energy (93 = 93; 78 vs 80, one more upload happened in V2). It did reduce deferral events by 33 % in S1 and S3 (9→6) and by 12.5 % in S2 (200→175),

because skip-and-fit removes head-of-line blocking. Its main gain is **critical data**: in S2 all three critical items were on time (V1: 2/3) and their mean delay fell from 5.0 to 0.67 ticks, and one more item was delivered (18 vs 17). In S3 the third critical item arrived after consent was revoked and was rightly rejected under both versions, so 2/3 is the correct maximum. Conclusion: the priority rule reorders work rather than removing it; it improves *what* is delivered first, not *how much* work is needed.

TASK B2 – Engineer an Adversarial Workload

Both workloads have **30 items** and run in the **same environment** with the same thresholds (36 ticks: network good for ticks 0–7, poor 8–15, offline 16–21, good from 22; Doze ticks 10–21; battery falls from 90 % by 2.5 per tick, below the 20 % floor at ticks 28–29, charging from tick 30; 3 workers max).

- **Workload A (easy):** arrivals only inside the good-network windows (ticks 0–7 and 22–33); sizes 1–3; 10 % critical (3 items).
- **Workload B (adversarial):** all arrivals inside the bad window (ticks 8–21, poor or no network, Doze); sizes 3–7; 30 % critical (14 items).

Prediction before running. B will need far more deferrals and will leave a backlog. Failed or blocked attempts will appear early (ticks 8–21) because items larger than the poor-link budget of 3 units cannot start and non-critical items are blocked by Doze. Pruning will therefore happen early and repeatedly for B, but only occasionally and briefly for A. I also expect that B cannot fully drain after the network returns, because total size (about 150 units) exceeds what 8 units per tick can carry in the remaining ticks.

Measured (priority scheduler V2; FIFO in brackets):

Measure	**Workload A**	**Workload B**
Items enqueued	30 (30)	30 (30)
Deferral events (distinct items)	11 (6) [15 (11)]	465 (28) [515 (30)]

Successful deliveries	30 [30]	21 [16]
Retries after failure	0	1
Final backlog	0 [0]	9 [14]
Critical items on time	3/3 [3/3]	1/14 [0/14]
Mean critical latency (ticks)	0.0 [0.67]	10.36 [17.2]
Energy (units)	86	126
Peak concurrent workers	3	2

The prediction held: B produced **42 times** more deferral events (465 vs 11), delivered 9 fewer items, used 47 % more energy (126 vs 86, wasted on the retried attempt and larger items) and missed 13 of 14 deadlines. Peak workers were lower in B (2 vs 3), because few items fit the budget at the same time, which shows that concurrency is not a measure of effort.

Investigation – why input size alone does not describe the effort. Both runs have n = 30, yet the work differs by a factor of 42. The effort depends on the *structure* of the instance: (1) **arrival time correlated with constraint windows**: B's arrivals all fall where the network is poor or absent and Doze is on, so every item waits through many ticks and is re-tested each tick; (2) **item size against the link budget**: sizes 3–7 against a 3-unit poor-link budget (and 8 on good links) create a bin-packing effect, so few items fit per tick; (3) **demand exceeds the capacity of the next window**: about 150 units are waiting against about 8 units × 14 good ticks = 112, so a backlog is unavoidable; (4) **critical deadlines** (3 ticks) expire during the outage, so the urgent items are the ones that fail. The structural property that made B hard is therefore the *overload of the recovery window caused by arrivals clustered in the constrained window*, not the count of items.

TASK B3 – Exact vs Approximate UI / Graphics Optimisation

Model (simplified feed renderer). A configuration is (quality q ∈ 1..L per item class, cache size c ∈ 1..C, prefetch distance p ∈ 0..P). Frame time = 7 ms base + miss-rate × decode time, where decode time = d · q^1.3 · device-slowdown s, the hit rate = min(1, c / working-set W) and each

prefetch step hides 22 % of the remaining misses. Cost = 100 · frame-drop fraction + 3 · λ · energy + 40 · quality loss, with λ = 1 + (100 − battery %) / 50 (a low battery makes energy dearer).

- **Solution A (exact):** enumerates every configuration and keeps the minimum cost.
- **Solution B (adaptive greedy, the A5.2 design):** starts at q = L/2, c = 1, p = 0. While p95 frame time > 16 ms it takes the cache or prefetch step with the best milliseconds saved per unit of added cost (quality is lowered only when cache and prefetch are exhausted or the battery is below 20 %). When the frame budget is met it tries a quality step, alone or combined with one cache or prefetch step, and keeps it only if the frame stays ≤ 16 ms and cost falls. It stops when no move helps (steady state).

B3.1 Correctness on Small Instances

L = 4, C = 4, P = 3 (64 configurations). The three instances were fixed before running and include a case where the heuristic is wrong.

Instance	**Parameters**	**Exact optimum (q, c, p)**	**Exact cost**	**Greedy result**	**Greedy cost**	**Match ?**
Flagship	s 1.0, W 3, scroll v 1.0, battery 80 %, d 2.2	(4, 1, 0)	24.27	(4, 1, 0)	24.27	Yes
Mid-range	s 1.8, W 4, v 1.5, battery 50 %, d 2.6	(4, 3, 0)	40.67	(2, 1, 0)	43.45	**No** (+6.8 %)
Low-end, low battery	s 3.0, W 4, v 2.0, battery 15 %, d 2.6	(1, 1, 0)	52.15	(1, 1, 0)	52.15	Yes

Over 500 further random instances the greedy cost equalled the exact cost in 420 (84 %); the mean cost excess was 0.33 and the worst excess 14.0.

B3.2 Counter-example and Trace

The mid-range instance above is the counter-example. Frame time and cost of the relevant configurations:

Configuration (q, c, p)	Frame time (ms)	Cost
(2, 1, 0) – greedy start and end	15.6	43.45
(3, 1, 0)	21.6	74.01
(3, 2, 0)	16.8	46.51
(3, 1, 1)	18.4	61.98
(4, 2, 0)	21.2	70.09
(4, 3, 0) – exact optimum	14.1	40.67

Trace. Start (2, 1, 0): frame time 15.6 ms ≤ 16, so no repair is needed and the loop goes to the quality phase. Candidates: q+1 → 21.6 ms (rejected), q+1 with cache+1 → 16.8 ms (rejected), q+1 with prefetch+1 → 18.4 ms (rejected). No move is accepted, so it stops at steady state with cost 43.45. **The decision that led away from the optimum** is stopping at the first feasible point and judging only one-step neighbours: the optimum needs a cache of 3 (two cache steps) together with quality 4, and every intermediate point (such as (3, 2, 0) at 16.8 ms) violates the 16 ms limit, so the path to it is blocked by an infeasible valley. In 4,000 random trials the worst instance (s 2.52, W 4, battery 100 %) gave exact cost 20.33 at (4, 3, 0) against greedy 35.06 at (2, 1, 0), 72 % higher.

B3.3 Increase the Problem Size

K item classes, L quality levels per class, C = 4, P = 3. Counts are operations, not wall-clock time.

K classes	L levels	Exact: configurations evaluated	Greedy: evaluations	Greedy: quality changes	Greedy: steps	Exact cost	Greedy cost	Greedy excess

1	3	48	9	2	3	31.27	31.27	0 %
2	4	256	7	0	1	44.74	45.13	+0.9 %
3	5	2,000	23	1	3	48.94	51.60	+5.4 %
4	6	20,736	19	0	3	49.90	58.57	+17.4 %
5	8	524,288	25	0	4	49.90	70.89	+42.1 %

The exact search grows as L^K · C · P (up to 524,288 evaluations) while the greedy tuner needs 7–25, about 20,000 times fewer at K = 5. Cache evictions are driven only by cache changes; in these runs greedy never reduced the cache, so there were 0 evictions, while the exhaustive search would imply a full cache rebuild for every candidate tried.

B3.4 Engineering Trade-off

Gained: a bounded decision cost (under 30 evaluations, at most 4 adaptation steps) that is independent of the size of the exact search space, no offline per-device search, and the ability to react as battery and content change. **Sacrificed:** the optimality guarantee; the heuristic can stop in a feasible but costlier configuration, and in this model the gap grew with problem size (0 % at K = 1, 42 % at K = 5) and in 16 % of the small random instances. **Where I would accept it:** a live feed on a population of unknown devices, where the real requirement is 16 ms frames now and conditions change every few seconds. I would still run the exact search offline in the lab for a few device tiers to choose good starting parameters and to measure the greedy gap before shipping.

TASK B4 – Final Engineering Decision: Multimedia Framework Selection (5 Marks)

B4.1 Practical Strategy

The app records a 15 s clip. Model: encoding energy = 2.5 W × encode time (the encode time is lower on faster CPUs and higher for larger resolutions); upload energy = radio power (Wi-Fi 0.8 W, LTE 2.0 W, poor LTE 2.5 W) × upload time; a deferred upload runs later on Wi-Fi or charger and is charged 30 % to the battery. Android realisation: `MediaCodec` (hardware H.264/HEVC

encoder) for encoding and WorkManager with NETWORK_UNMETERED or NETWORK_CONNECTED for the upload; on iOS, AVFoundation with Core Media and Core Video buffers and a URLSession background upload.

Selection rule (greedy, resolution first):

1. Energy budget per clip: 0.40 % of the battery if battery > 50 %, 0.20 % if 21–50 %, 0.08 % if ≤ 20 %; no limit when charging on Wi-Fi.
2. For resolution from 1080p down to 360p, and for bitrate from high to low, choose the first option whose energy fits the budget **and** whose total time is ≤ 60 s: upload now.
3. If nothing fits, choose the best resolution whose encode energy plus the deferred Wi-Fi upload fits the budget, and **defer** the upload until Wi-Fi or charging.

Quality score = resolution score (1.0, 1.6, 2.6, 3.6) × (0.7 + 0.15 × bitrate level). The exhaustive baseline minimises objective = −10 × quality + 40 × battery% + 0.01 × latency(s) over all 4 resolutions × 3 bitrates × 2 timings = 24 options, under the same hard constraints.

Output for six sample devices:

Device profile	**Chosen resolution @ bitrate**	**Upload**	**Est. energy**	**Est. time**	**Battery used by clip**	**Battery left**	**Same as exhaustive?**
Flagship, Wi-Fi, charging (90 %)	1080p @ 8.0 Mbps	now	43.2 J	20.0 s	0.069 %	89.93 %	Yes
Flagship, good LTE (70 %)	1080p @ 8.0 Mbps	now	70.0 J	31.0 s	0.112 %	69.89 %	Yes
Mid-range, LTE (45 %)	1080p @ 5.0 Mbps	now	97.5 J	42.8 s	0.176 %	44.82 %	Yes
Mid-range, poor LTE 1 Mbps (60 %)	720p @ 2.8 Mbps	now	135.0 J	54.0 s	0.244 %	59.76 %	**No**

Low-end, LTE, low battery (15 %)	360p @ 1.2 Mbps	deferred to Wi-Fi	28.7 J	12.0 s	0.068 %	14.93 %	Yes
Low-end, poor LTE 0.8 Mbps (40 %)	360p @ 0.9 Mbps	now	70.3 J	28.1 s	0.169 %	39.83 %	**No**

B4.2 Challenge the Strategy

The mid-range phone on poor LTE (CPU 1.0, battery 60 %, 1 Mbps uplink) breaks the rule. The heuristic uploads 720p at 2.8 Mbps now (objective −11.82, 54 s on a radio at 2.5 W). The exhaustive search finds 1080p at 8 Mbps with the upload **deferred** to Wi-Fi (objective −22.35), a better clip for less battery drawn from the radio. The low-end phone on poor LTE shows the same pattern (−1.45 against −9.43 for 720p deferred). Cause: the rule is resolution-first and treats "upload now" as the default, so it never compares a lower-quality immediate upload with a high-quality deferred one. The heuristic matched the exact answer on 4 of 6 devices and was worse by 10.5 and 8.0 objective units on the other two.

B4.3 Production Decision Note

Decision: ship the greedy rule with a deferral-first fix for poor links, behind a staged rollout.

- **Why exhaustive search is unattractive at production scale.** Real choices add codec profile, frame rate, keyframe interval, thermal state and upload window to resolution and bitrate, so the space multiplies, and it must be solved per clip on thousands of concurrent devices whose network changes while the search runs. Even our toy 24-option search needs an energy model evaluated 24 times per clip on the device or a server round trip.
- **Guarantee provided.** Feasibility only: every chosen option respects the battery budget and the 60 s limit, or is deferred. There is no optimality guarantee; the six-device test shows a gap on 2 devices, both on poor links.
- **Advantage.** At most 12 candidate checks (against 24) and no network call, so the decision is instant, works offline and costs almost no energy; the result is deterministic and easy to explain.

- **When exact or cloud assistance is appropriate.** Offline, to calibrate thresholds per device tier; for a fleet-wide policy computed on the server; and for cloud-assisted transcoding when the device is on Wi-Fi and charging, where the phone uploads a higher bitrate and the server produces the delivery renditions.
- **Evidence to monitor before full deployment.** Upload success rate and retry rate; p95 time from capture to delivery; battery drop per clip by device tier; percentage of clips deferred and age of the oldest deferred clip; encoder failures and thermal throttling events; ANR and crash rate; server bandwidth cost. Roll out to 1 %, then 10 %, then 50 %, with automatic rollback if the success rate or the battery per clip degrades beyond the pre-agreed threshold.

Summary and Conclusions

The constraint-aware design (A1, B1) delivered all critical data on time in the stressed scenario, and the priority order mainly improved **which** work is done first (mean critical delay 5.0 → 0.67 ticks) rather than the total work. The adversarial workload (B2) showed 42× more deferrals for the same input size, so effort depends on instance structure. In UI and multimedia tuning (B3, B4), greedy strategies were about 20,000 times cheaper than exhaustive search but gave up optimality, in 16 % of small cases and increasingly at larger sizes. The practical engineering answer is to use the cheap adaptive rule on the device, keep exact search for offline calibration, and monitor delivery, battery and latency before widening the rollout.